Understand data models

Modern solutions handle diverse data, such as transactions, events, documents, telemetry, binary assets, and analytical facts. A single data store rarely satisfies all access patterns efficiently. Most production systems adopt *polyglot persistence*, which means that you select multiple storage models. This article centralizes the canonical definitions of the primary data store models available on Azure and provides comparative tables to accelerate model selection before you choose specific services.

Use the following steps to select your data models:

1. Identify workload access patterns, such as point reads, aggregations, full-text, similarity, time-window scans, and object delivery.
2. Map patterns to the storage models in the following sections.
3. Create a shortlist of Azure services that implement those models.
4. Apply evaluation criteria, such as consistency, latency, scale, governance, and cost.
5. Combine models only where access patterns or life cycles clearly diverge.

How to use this guide

Each model section includes a concise definition, typical workloads, data characteristics, example scenarios, and links to representative Azure services. Each section also includes a table to help you choose the right Azure service for your use case. In some cases, you can use other articles to make more informed choices about Azure service options. The respective model sections reference those articles.

Two comparative tables summarize nonrelational model traits to help you quickly evaluate options without repeating content across sections.

Classification overview

Expand table

Category	Primary purpose	Typical Azure service examples
Relational (OLTP)	Consistent transactional operations	Azure SQL Database, Azure Database for PostgreSQL, or Azure Database for MySQL
Nonrelational, such as document, key-value, column-family, and graph	Flexible schema or relationship-centric workloads	Azure Cosmos DB APIs, Azure Managed Redis, Managed Cassandra, or HBase
Time series	High-ingest timestamped metrics and events	Azure Data Explorer or Eventhouse in Fabric
Object and file	Large binary or semi-structured file storage	Azure Blob Storage or Azure Data Lake Storage
Search and indexing	Full-text and multi-field relevance, secondary indexing	Azure AI Search
Vector	Semantic or approximate nearest neighbor (ANN) similarity	Azure AI Search or Azure Cosmos DB variants
Analytics, online analytical processing (OLAP), massively parallel processing (MPP)	Large-scale historical aggregation or business intelligence (BI)	Microsoft Fabric, Azure Data Explorer, Azure Analysis Services, or Azure Databricks

> **Note**
>
> A single service might provide multiple models, also known as *multimodel*. Choose the best-fit model instead of combining models in a way that complicates operations.

Relational data stores

Relational database management systems organize data into normalized tables by using schema-on-write. They enforce integrity and support atomicity, consistency, isolation, and durability (ACID) transactions and rich SQL queries.

Strengths: Multi-row transactional consistency, complex joins, strong relational constraints, and mature tooling for reporting, administration, and governance.

Considerations: Horizontal scale generally requires sharding or partitioning, and normalization can increase join cost for read-heavy denormalized views.

Workloads: Order management, inventory tracking, financial ledger recording, billing, and operational reporting.

Select an Azure service for relational data stores

- SQL Database is a managed relational database for modern cloud applications that use the SQL Server engine.
- Azure SQL Managed Instance is a near-complete SQL Server environment in the cloud that's ideal for lift-and-shift migrations.
- SQL Database (Hyperscale) is a highly scalable SQL tier designed for massive workloads with fast autoscaling and rapid backups.
- Azure Database for PostgreSQL is a managed PostgreSQL service that supports open-source extensions and flexible deployment options.
- Azure Database for MySQL is a managed MySQL database for web apps and open-source workloads.
- SQL Database in Fabric is a developer-friendly transactional database, based on SQL Database, that you can use to easily create an operational database in Fabric.

Use the following table to help determine which Azure service meets your use case requirements.

Expand table

Service	Best for	Key features	Example use case
SQL Database	Cloud-native apps	Managed, elastic pools, Hyperscale, built-in high availability, advanced security	Building a modern software as a service (SaaS) application by using a scalable SQL back end
SQL Managed Instance	Legacy enterprise apps	Full SQL Server compatibility, lift-and-shift support, virtual networks, advanced auditing	Migrating an on-premises SQL Server app by using minimal code changes
SQL Database (Hyperscale)	Global distribution	Multi-region read scalability, geo-replication, rapid autoscaling	Serving a globally distributed app that requires high read throughput

Service	Best for	Key features	Example use case
Azure Database for PostgreSQL	Open-source, analytics workloads	PostGIS, hyperscale, elastic clusters, open-source extensions	Developing a geospatial analytics app by using PostgreSQL and PostGIS
Azure Database for MySQL	Lightweight web apps	Flexible Server, open-source compatibility, cost-effective	Hosting a WordPress-based e-commerce site
SQL Database in Fabric	Online transaction processing (OLTP) workloads in the Fabric ecosystem	Built on the SQL Database engine, scalable, and integrated into Fabric	Building AI apps on an operational, relational data model that includes native vector search capabilities

Nonrelational data stores

Nonrelational databases, also called *NoSQL databases*, optimize for flexible schemas, horizontal scale, and specific access or aggregation patterns. They typically relax some aspects of relational behavior, such as schema rigidity and transaction scope, for scalability or agility.

Document data stores

Use document data stores to store semi-structured documents, often in JSON format, where each document includes named fields and data. The data can be simple values or complex elements, such as lists and child collections. Per-document schema flexibility enables gradual evolution.

Strengths: Natural application object mapping, denormalized aggregates, multi-field indexing

Considerations: Document size growth, selective transactional scope, need for careful data shape design for high-scale queries

Workloads: Product catalogs, content management, profile stores

Select an Azure service for document data stores

- Azure Cosmos DB for NoSQL is a schema-less, multi-region NoSQL database that has low-latency reads and writes.
- Azure DocumentDB is a globally distributed database that has MongoDB wire protocol compatibility and autoscaling.
- Azure Cosmos DB in Fabric is a schema-less, NoSQL database that has low-latency reads and writes, simplified management, and built-in Fabric analytics.

Use the following table to help determine which Azure service meets your use case requirements.

Expand table

Service	Best for	Key features	Example use case
Azure Cosmos DB for NoSQL	Custom JSON document models that support SQL-like querying	Rich query language, multi-region writes, time to live (TTL), change feed	Building a multitenant SaaS platform that supports flexible schemas
Azure DocumentDB	Apps that use MongoDB drivers or JSON-centric APIs	Global distribution, autoscale, MongoDB-native wire protocol	Migrating a Node.js app from MongoDB to Azure
Azure Cosmos DB in Fabric	Real-time analytics over NoSQL data	Automatic extract, transform, and load (ETL) to OneLake through Fabric integration	Transactional apps that include real-time analytical dashboards

Column-family data stores

A column-family database, also known as a *wide-column database*, stores sparse data into rows and organizes dynamic columns into column families to support co-access. Column orientation improves scans over selected column sets.

Strengths: High write throughput, efficient retrieval of wide or sparse datasets, dynamic schema within families

Considerations: Up-front row key and column family design, secondary index support varies, query flexibility lower than relational

Workloads: Internet of Things (IoT) telemetry, personalization, analytics preaggregation, time-series style tall data when you don't use a dedicated time-series database

Select an Azure service for column-family data stores

- Azure Managed Instance for Apache Cassandra is a managed service for open-source Apache Cassandra clusters.
- Apache HBase on Azure HDInsight is a scalable NoSQL store for big data workloads built on Apache HBase and the Hadoop ecosystem.
- Azure Data Explorer (Kusto) is an analytics engine for telemetry, logs, and time-series data that uses Kusto Query Language (KQL).

Use the following table to help determine which Azure service meets your use case requirements.

Expand table

Service	Best for	Key features	Example use case
Azure Managed Instance for Apache Cassandra	New and migrated Cassandra workloads	Managed, native Apache Cassandra	IoT device telemetry ingestion that supports Cassandra compatibility
Apache HBase on HDInsight	Hadoop ecosystem, batch analytics	Hadoop Distributed File System (HDFS) integration, large-scale batch processing	Batch processing of sensor data in a manufacturing plant
Azure Data Explorer (Kusto)	High-ingest telemetry, time-series analytics	KQL, fast ad-hoc queries, time-window functions	Real-time analytics for application logs and metrics

Key-value data stores

A key-value data store associates each data value with a unique key. Most key-value stores only support simple query, insert, and delete operations. To modify a value either partially or completely, an application must overwrite the existing data for the entire value. In most implementations, reading or writing a single value is an atomic operation.

Strengths: Simplicity, low latency, linear scalability

Considerations: Limited query expressiveness, redesign needed for value-based lookups, large value overwrite cost

Workloads: Caching, sessions, feature flags, user profiles, recommendation lookups

Select an Azure service for key-value data stores

- Azure Managed Redis is a managed in-memory data store based on the latest Redis Enterprise version that provides low latency and high throughput.
- Azure Cosmos DB for Table is a key-value store optimized for fast access to structured NoSQL data.
- Azure Cosmos DB for NoSQL is a document data store that's optimized for fast access to structured NoSQL data and provides horizontal scalability.

Use the following table to help determine which Azure service meets your use case requirements.

Expand table

Service	Best for	Key features	Example use case
Azure Managed Redis	High-speed caching, session state, publish-subscribe	In-memory store, submillisecond latency, Redis protocol	Caching product pages for an e-commerce site
Azure Cosmos DB for Table	Migrating existing Azure Table Storage workloads	Table Storage API compatibility	Storing user preferences and settings in a mobile app
Azure Cosmos DB for NoSQL	High-speed caching with massive scale and high availability	Schema-less, multi-region, autoscale	Caching, session state, serving layer

Graph data stores

A graph database stores information as nodes and edges. Edges define relationships, and both nodes and edges can have properties similar to table columns. You can analyze connections between entities, such as employees and departments.

Strengths: Relationship-first querying patterns, efficient variable-depth traversals

Considerations: Overhead if relationships are shallow, requires careful modeling for performance, not ideal for bulk analytical scans

Workloads: Social networks, fraud rings, knowledge graphs, supply chain dependencies

Select an Azure service for graph data stores

Use SQL Server graph extensions for storing graph data. The graph extension extends the capabilities of SQL Server, SQL Database, and SQL Managed Instance to enable modeling and querying complex relationships by using graph structures directly within a relational database.

Time-series data stores

Time-series data stores manage a set of values organized by time. They support features like time-based queries and aggregations. They're optimized to ingest and analyze large volumes of data in near real time. They're typically append-only databases.

Strengths: Compression, high-volume ingestion, time-window queries and aggregations, out-of-order ingestion handling

Considerations: Tag cardinality management, retention cost, downsampling strategy, specialized query languages

Workloads: IoT sensor metrics, application telemetry, monitoring, industrial data, and financial market data

Select an Azure service for time-series data stores

- Azure Data Explorer is a managed big data storage platform. Use it to query and visualize high volumes of data in near real time. Choose this service if you need a standalone platform as a service (PaaS) solution with granular control over cluster configuration, networking, and scaling.
- Eventhouse in Microsoft Fabric is part of the Real-Time Intelligence experience in Fabric. It uses KQL databases to handle streaming data. Choose this service if you want a software as a service (SaaS) experience that's integrated with the Fabric ecosystem, including OneLake and other Fabric workloads.
- Some transactional databases provide limited time-series capabilities as part of their broader feature set or through extensions. For example, Azure Database for PostgreSQL supports TimescaleDB. Select this option if you need to query time-series data alongside existing transactional data in the database.

When you choose a time-series data store, evaluate the service based on your workload's needs for:

- Ingestion performance
- Ad-hoc queries
- Additional indexes beyond date/time fields
- Time-series analytics and alerts

Object data stores

Store large binary or semi-structured objects and include metadata that rarely changes or remains immutable.

Strengths: Virtually unlimited scale, tiered cost, durability, parallel read capability

Considerations: Whole-object operations, metadata query limited, eventual listing behaviors

Workloads: Media assets, backups, data lake raw zones, log archives

Select an Azure service for object data stores

- Data Lake Storage is a big data-optimized object store that combines hierarchical namespace and HDFS compatibility for advanced analytics and large-scale data processing.
- Blob Storage is a scalable object store for unstructured data like images, documents, and backups that includes tiered access for cost optimization.

Use the following table to help determine which Azure service meets your use case requirements.

Expand table

Service	Best for	Key features	Example use case
Data Lake Storage	Big data analytics and hierarchical data	HDFS, hierarchical namespace, optimized for analytics	Storing and querying petabytes of structured and unstructured data by using Azure Data Factory or Azure Databricks
Blob Storage	General-purpose object storage	Flat namespace, simple REST API, and tiered storage that includes hot, cool, and archive	Hosting images, documents, backups, and static website content

Search and indexing data stores

A search engine database allows applications to search for information in external data stores. A search engine database can index massive volumes of data and provide near real-time access to these indexes.

Strengths: Full-text queries, scoring, linguistic analysis, fuzzy matching

Considerations: Eventual consistency of indexes, separate ingestion or indexing pipeline, cost of large index updates

Workloads: Site or product search, log search, metadata filtering, multi-attribute discovery

Select an Azure service for search data stores

For more information, see Choose a search data store in Azure.

Vector search data stores

Vector search data stores store and retrieve high-dimensional vector representations of data, often generated by machine learning models.

Strengths: Semantic search, ANN algorithms

Considerations: Indexing complexity, storage overhead, latency versus accuracy, integration challenges

Workloads: Semantic document search, recommendation engines, image and video retrieval, fraud and anomaly detection

Select an Azure service for vector search data stores

For more information, see Choose an Azure service for vector search.

Analytics data stores

Analytics data stores store big data and persist it throughout an analytics pipeline life cycle.

Strengths: Scalable compute and storage, support for SQL and Spark, integration with BI tools, time-series and telemetry analysis

Considerations: Cost and complexity of orchestration, query latency for ad-hoc workloads, governance across multiple data domains

Workloads: Enterprise reporting, big data analytics, telemetry aggregation, operational dashboards, data science pipelines

Select an Azure service for analytics data stores

For more information, see Choose an analytical data store in Azure.

Comparative characteristics (core nonrelational models)

Expand table

Aspect	Document	Column-family	Key-value	Graph
Normalization	Denormalized	Denormalized	Denormalized	Normalized relationships
Schema approach	Schema on read	Column families defined, columns schema on read	Schema on read	Schema on read
Consistency (typical)	Tunable for each item	For each row or family	For each key	For each edge or traversal semantics

Aspect	Document	Column-family	Key-value	Graph
Atomicity scope	Document	Row or family, depending on table implementation	Single key	Graph transaction (varies)
Locking and concurrency	Optimistic (ETag)	Pessimistic or optimistic, depending on implementation	Optimistic (key)	Optimistic (pattern)
Access pattern	Aggregate (entity)	Wide sparse aggregates	Point lookup by key	Relationship traversals
Indexing	Primary and secondary	Primary and limited secondary	Primary (key)	Primary and sometimes secondary
Data shape	Flexible hierarchical	Sparse tabular wide	Opaque value	Nodes and edges
Sparse/Wide suitability	Yes/Yes	Yes/Yes	Yes/No	No/No
Typical datum size	Small–medium	Medium–large	Small	Small
Scale dimension	Partition count	Partition and column family width	Key space	Node or edge count

Comparative characteristics (specialized nonrelational models)

Expand table

Aspect	Time series	Object (blob)	Search/Indexing
Normalization	Normalized	Denormalized	Denormalized
Schema	Schema on read (tags)	Opaque value and metadata	Schema on write (index mapping)
Atomicity scope	N/A (append)	Object	For each document or index operation
Access pattern	Time-slice scans, window aggregation	Whole-object operations	Text queries and filters
Indexing	Time and optional secondary	Key (path) only	Inverted and optional facets

Aspect	Time series	Object (blob)	Search/Indexing
Data shape	Tabular (timestamp, tags, value)	Binary or blob with metadata	Tokenized text and structured fields
Write profile	High-rate append	Bulk or infrequent updates	Batch or streaming index
Read profile	Aggregated ranges	Whole or partial downloads	Ranked result sets
Growth driver	Event rate multiplied by retention	Object count and size	Indexed document volume
Consistency tolerance	Eventual for late data	Read-after-write for each object	Eventual for new documents

Choose among models (heuristics)

Expand table

Need	Prefer
Strict multi-entity transactions	Relational
Evolving aggregate shape, JSON-centric APIs	Document
Extreme low-latency key lookups or caching	Key-value
Wide, sparse, write-heavy telemetry	Column family or time series
Deep relationship traversal	Graph
Massive historical analytical scans	Analytics or OLAP
Large unstructured binaries or lake zones	Object
Full-text relevance and filtering	Search and indexing
High-ingest timestamp metrics with window queries	Time series
Rapid similarity (semantic or vector)	Vector search

Combine models and avoid pitfalls

Use more than one model when the following scenarios apply:

- Access patterns diverge, such as point lookup versus wide analytical scan versus full-text relevance.

- Life cycle and retention differ, such as immutable raw versus curated structured.
- Latency versus throughput requirements conflict.

Avoid premature fragmentation:

- Use one service when it still meets performance, scale, and governance objectives.
- Centralize shared classification logic, and avoid duplicate transformation pipelines across stores unless necessary.

Watch for the following common antipatterns:

- Multiple microservices share one database, which creates coupling.
- Teams add another model without operational maturity, such as monitoring or backups.
- A search index becomes the primary data store, which leads to misuse.

When to re-evaluate your model choice

Expand table

Signal	Possible action
Increasing ad-hoc joins on a document store	Introduce relational read model
High CPU on search index because of analytical aggregations	Offload to analytics engine
Large denormalized documents create partial update contention	Reshape aggregates or split
Time-window queries slow on column-family store	Adopt purpose-built time-series database
Point lookup latency rises with graph traversal depth	Add derived materialized views

Next steps

- The Secure methodology in the Cloud Adoption Framework for Azure
- Zero Trust framework data security

Related resources

- OLAP guidance
- OLTP guidance
- Data lakes overview

- ETL overview

Use the following articles to choose a specialized data store:

- Choose a big data storage technology in Azure
- Choose a search data store in Azure
- Choose an Azure service for vector search

Learn about reference architectures that use the Azure services in this article:

- The baseline highly available zone-redundant web application architecture uses SQL Database as its relational data store.
- The deploy microservices with Azure Container Apps and Dapr architecture uses SQL Database, Azure Cosmos DB, and Azure Managed Redis as data stores.
- The automate document classification in Azure architecture uses Azure Cosmos DB as its data store.

Last updated on 09/23/2025